

Implementation Analysis

Looking at your ObdTestApp project against the PDF specification, here's how the implementation aligns:

✓ Well-Implemented Components

1. Transport Abstraction (Document §7.1)

The IElmTransport interface matches the document's design exactly:

// Document specifies: // Your implementation:

OpenAsync ✓ OpenAsync

ReadAsync ✓ ReadAsync

WriteAsync ✓ WriteAsync

FlushAsync ✓ FlushAsync

IsOpen ✓ IsOpen

You've extended beyond the document with three transport implementations: BleElmTransport, BtElmTransport, and the document's serial example.

2. Read-Until-Prompt Framing (Document §3, §7.2)

ElmFramer.SendAndReadFrameAsync correctly implements:

- CR-terminated command transmission
- Reading until > prompt (0x3E)
- Dropping NULL bytes (0x00) as the datasheet recommends
- Timeout via linked CancellationTokenSource

3. Baseline Initialization (Document §4)

ElmSession.BaselineInitAsync follows the exact command sequence:

AT Z → AT D → AT E0 → AT L0 → AT S0 → AT H0 → AT AT1

4. Protocol Detection & Locking (Document §4)

DetectAndLockProtocolAsync correctly:

- Sets AT SP 0 for auto-search

- Probes with 0100
- Reads AT DPN, strips the 'A' prefix
- Locks with AT SP {protocol}
- Verifies with another 0100 probe

5. Recovery Ladder (Document §6)

RecoverAsync implements all four levels:

Level	Document	Implementation
0	Parser resync (send CR, drain)	TryResyncAsync - sends empty command
1	AT PC + reapply protocol	TryProtocolResyncAsync ✓
2	Baseline + locked protocol	✓ Inline in RecoverAsync
3	AT Z + full re-detect	Calls BaselineInitAsync + DetectAndLockProtocolAsync

⚠ Gaps and Deviations

1. No Independent Health Checks

The document (§5) recommends periodic health checks independent of query activity:

"a periodic health check (re-sending a known-good PID like 0100 or 010C) detects silent failure modes early"

Your implementation only triggers recovery reactively after MaxConsecutiveFailures. There's no background watchdog.

2. Recovery Only Triggers After N Failures, Not Per-Query

In QueryAsync:

```
if (_failures >= MaxConsecutiveFailures)
{
    await RecoverAsync(ct);

    _failures = 0;
}
```

The document implies recovery should be attempted immediately on failure, with escalation across attempts. Your implementation waits for 3 consecutive failures before any recovery attempt.

3. Transport-Layer Reconnection Missing

Neither BleElmTransport nor BtElmTransport has reconnection logic. If `_isOpen` becomes false due to disconnection, all operations throw `InvalidOperationException` with no path back to a working state.

4. Level 0 Resync Implementation

```
private async ValueTask<bool> TryResyncAsync(Cancellation_token ct)
{
    var frame = await _framer.SendAndReadFrameAsync("", TimeSpan.FromSeconds(2), ct);
    return ElmParsing.NormalizeLines(frame).Length >= 0; // Always true!
}
```

Two issues:

- The condition `Length >= 0` is always true (arrays can't have negative length)
- The document recommends draining until prompt; this just sends one CR

5. No Metrics/Diagnostics

Document §8 recommends:

"Persist structured metrics: average/percentile round-trip-time, consecutive failures, recovery level entered"

No metrics instrumentation exists in the current implementation.

Summary Verdict

Aspect	Status
Core framing protocol	✓ Faithful
Initialization sequence	✓ Faithful

Aspect	Status
Protocol lock mechanism	✓ Faithful
Recovery ladder structure	✓ Present
Recovery ladder triggering	⚠ Delayed (waits for N failures)
Transport reconnection	✗ Missing
Independent health checks	✗ Missing
Metrics/observability	✗ Missing

The implementation captures the document's core algorithm but doesn't achieve the "self-stabilizing" property because transport failures are terminal and recovery is reactive rather than proactive.